

Decision Tree Classifier

Division of labor

We worked on most of the project together. The entirety of part 1, as well as the report was done together. Otherwise, Oskar did most of the work on Part 2, and Julia did most of the work on Part 3.

Introduction

This project describes the implementation of a decision tree, following the ID3-algorithm. The tree's performance was then tested on the "churn-data" dataset and was compared with the `DecisionTreeClassifier` from `scikit-learn`. Finally, the importance of the various features was evaluated using permutation importance. The project demonstrates the function of the implemented decision tree, as well as the general function of decision trees.

Part 1: Implementing the Decision tree classifier

The implementation of the Decision Tree model utilizes the ID3 algorithm, which builds the tree recursively starting from the root node, structuring the tree in a dictionary. The model is greedy and identifies the optimal split for the current node, maximizing information gain (IG). To measure the IG, the model supports the impurity measures "Entropy" and "Gini". The impurity measure gives information about how pure the class labels are within a node. The "Entropy" measure comes from *Stirling's Theorem*, and can be written as:

$$H(\text{Decision}) = - \sum_i P(\text{Decision} = i) \cdot \log_2(P(\text{Decision} = i))$$

"Gini" uses a *Taylor approximation* of the Entropy, eliminating logarithmic operations, and can be written as:

$$G(\text{Decision}) = \sum_i P(\text{Decision} = i) \cdot (1 - P(\text{Decision} = i))$$

where both Entropy and Gini are the sum over outcomes. Since the class label is binary consisting of 0 or 1, the purity measures can be calculated as:

$$H = -(p \cdot \log_2 p + (1 - p) \cdot \log_2(1 - p))$$

$$G = 2 \cdot p \cdot (1 - p)$$

where p is the ratio of true labels, calculated as sum of the array over the size of the array. Though this requires the class label to only consist of 0 and 1, it makes for a fast and simple calculation.

IG is calculated as the difference in impurity before versus after the split, weighted by the size of the resulting split. When splitting the data in a node to calculate the IG, the data can be split based on various thresholds. In this implementation, the IG is calculated on five different thresholds/splits, and the split that yields the highest IG is kept. Three of the five thresholds are the 25th, 50th (median) and 75th percentiles. The remaining thresholds are calculated dynamically, using the ratio of positive class labels, p . This reveals any natural separation of the two classes. Since it is unknown whether the top or bottom p percentiles yield the best split, both the p -th and $(100-p)$ -th percentiles of the features are checked. This does not result in any data leakage, as the percentiles are strictly calculated using the train data within the current node. The algorithm repeats this process for every feature present in the node, splitting on the combination of feature and threshold giving the highest IG. Since the dataset provided consisted only of numerical features, the algorithm does not support the multi-way categorical split commonly used in an ID3-tree.

To implement the ID3-algorithm, the model builds the tree recursively, starting with the root node containing all the training data. The model finds the best split for the node, looking for the highest IG. The node remembers the feature and threshold to split on for later predictions. It then performs the split, creating two subsets of the data. Both these subsets are treated as new nodes, and the process is repeated recursively. The recursive call stops at one of three base cases. The first case occurs when the maximum depth of the tree is reached, if specified in the constructor. Each recursive call knows its current depth in the tree and stops when the depth is equal to the maximum depth. The second case occurs when all class labels in a node are equal. Here, the node is pure and there is no need to split the data further. The third and last case occurs if all data points have identical feature values. Here, there is no way to distinguish the remaining data. When one of the three base cases is reached, the call returns a leaf with the most common label in the node.

Part 2: Model selection

In part 2, the implemented model was tested on the *churn-data* dataset, predicting whether customers would discontinue their subscription, *churn*, or not, *no churn*. When importing the dataset, it was split into an 80% training set and 20% testing set to maximize generalization and prevent any data leakage. The data was distributed pseudo-randomly by specifying random state, ensuring reproducible result. To ensure equal distribution of the class label, *stratify = y* was specified when

splitting. The train data was visualized by feature to get an overview of the data shown in figure 1. Since all values had been transformed to numeric values, histograms were used for this visualization.

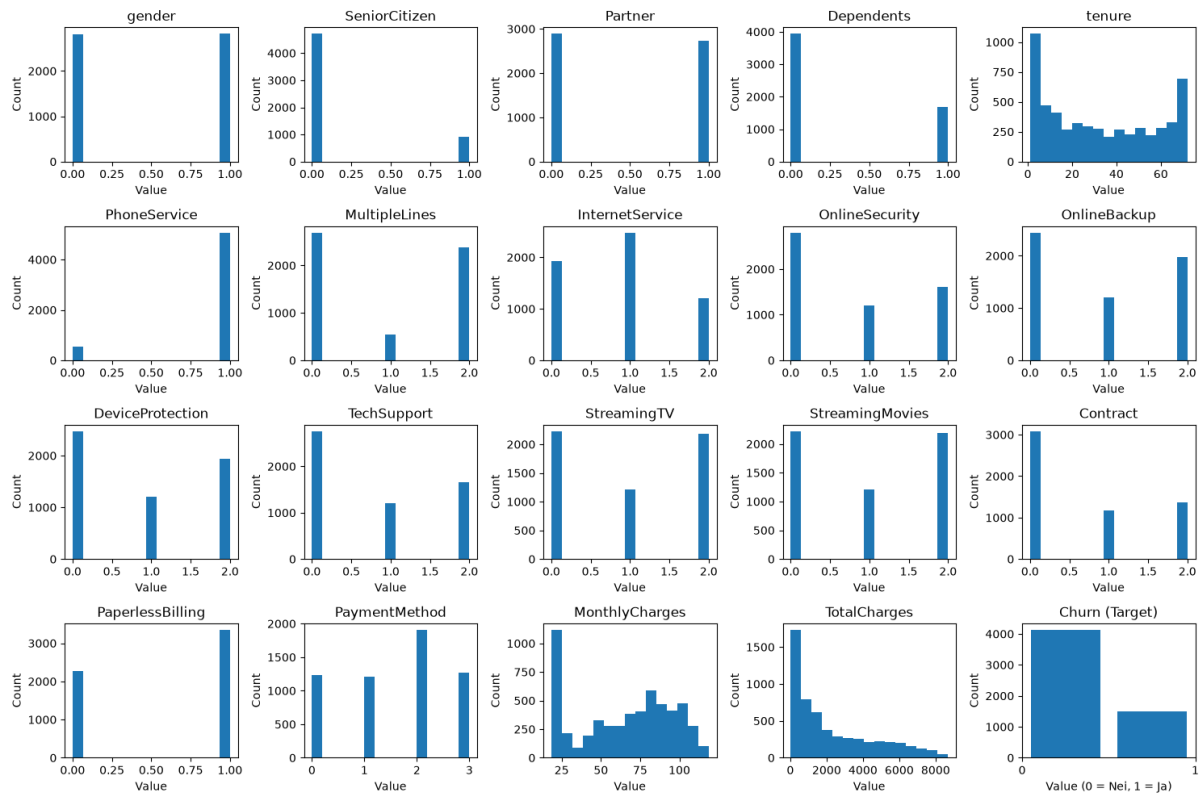


Figure 1: Distribution of the data features and the class label.

Figure 1 shows 16 categorical features and 3 continuous features, as well as the target feature containing far more no-churn than churn labels.

The Decision Tree supports the hyperparameters 'max_depth' and 'criterion'. In this model selection, the following values

$$\text{max_depth} \in \{1, 2, 3, 4, 5, 6, 7, 10, \text{None}\}$$

$$\text{criterion} \in \{\text{entropy}, \text{gini}\}$$

were tested, with *None* letting the tree grow until one of the other base cases is reached. Small increments testing shallow depths were chosen to check if shallow trees perform well on the data without underfitting. The moderate to high depths allows the tree to get more complex, and tests how prone the model is to overfit. The non-specified depth works as a baseline showing the highest complexity and possibly overfitting. Criterion was tested for both the supported purity measures.

When tuning the hyperparameters 'max_depth' and 'criterion', k-fold-cross-validation was used. This maximizes the amount of training data by training multiple models on different parts of the data, rotating the data used for validation. Given the relatively small dataset, a value of $k = 5$ was chosen to ensure a reliable balance between training and validation size. For every combination of

'max_depth' and 'criterion', a k-fold cross-validation was performed, keeping the combination which yields the highest mean performance on the validation data. To measure the performance of a model, the F1-score was used. The F1-score combines precision and recall, making it a suitable metric for the skewed data in the *churn-data* dataset. Figure 2 shows the F1-score on validation data for both the implemented ID3-model and Scikit-learns model.

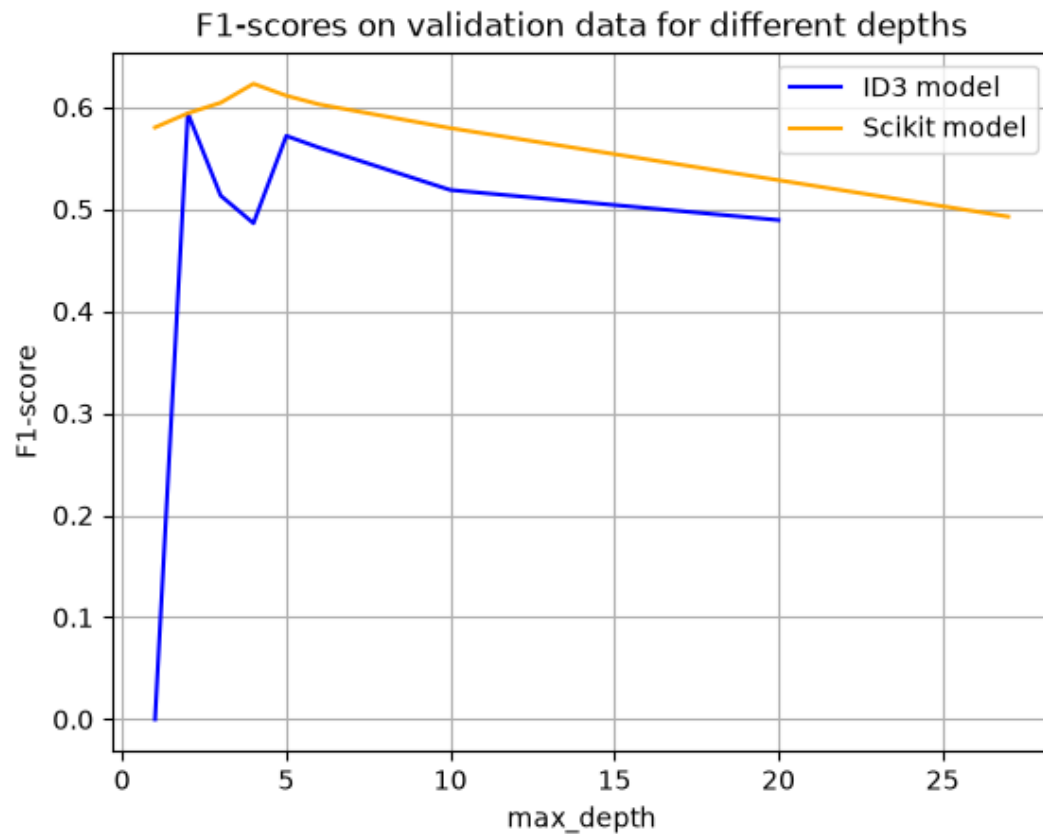


Figure 2: F1-score on validation data for different maximum depths.

Table 1: model performance with optimal hyperparameters, as well as unrestricted depth.

Model	max depth	criterion	F1 – train	F1 – val
ID3	2	Entropy	0,602	0,594
Scikit – learn	4	Gini	0,634	0,623
ID3	20 18 (None)	Entropy Gini	0,998 0,998	0,489 0,494
Scikit – learn	28 27 (None)	Entropy Gini	0,997 0,997	0,489 0,493

Table 1 shows the best combinations of hyperparameters for both models, as well as the maximum depths. The implemented ID3-model performs best with maximum depth at 2 and *entropy* as purity measure, and the Scikit-learn model performs best with maximum depth at 4 and *gini* as purity

measure. When letting the tree grow unrestricted, the implemented model stops at around 18-20 layers, whereas the Scikit-learn model keeps growing until a depth of 27-28. In both cases the model is extremely overfitted on the training data, as the F1-score on validation data is under half of that on the training data.

After finding the optimal hyperparameters from k-fold cross-validation, a new model is trained on the entirety of the train data. This makes for a more robust model, as the training size is larger compared to the cross-validation models. Lastly, the performance of the final model was evaluated using the test data. This pipeline was run for both the implemented ID3-tree and the DecisionTreeClassifier of scikit-learn. When using the Scikit-learn model, both random state and class weight were specified. Specified random state ensured reproducibility, and class weight set to *balanced* allowed the model to perform better on the smaller churn-class.

To get an even more robust understanding of performance, the final hyperparameters were tested using multiple random states. By splitting, training and testing on 10 different seeds (random states), the final performance is here defined as the mean testing score of the different splits, with a range being the standard deviation of these scores. The final performance was measured using both accuracy and the F1-score.

Table 2: Final model performance on unseen test data.

	<i>ID3 – model</i>	<i>Scikit – learn – model</i>
<i>F1 – test</i>	$0,59 \pm 0,02$	$0,61 \pm 0,01$
<i>Accuracy – test</i>	$0,74 \pm 0,01$	$0,75 \pm 0,01$

As presented in table 2, the Scikit-learn model performs slightly better than the ID3-model in terms of both F1- and accuracy-metrics. Both models perform considerably higher on accuracy than on F1-score. This is expected from our skewed data, with F1 score prioritizing the rarer churn-label by calculating both precision and recall. Overall, both models perform consistently across different seeds, expressed by the low standard deviations.

The performance of the models can be visualized in a confusion matrix, visualizing how well and how often the model predicts the different labels. The confusion matrix for both models is represented in figure 3.

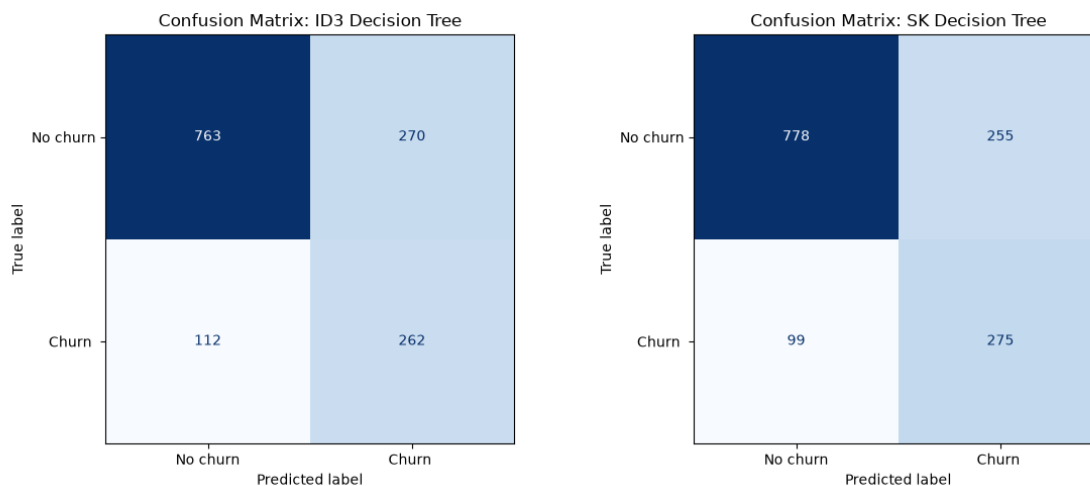


Figure 3: Confusion matrix for the ID3 model and the scikit-learn model.

As shown in figure 3, both models identify the no churn label well. When the true label is churn, both models predict the true label 2-3 times as often as the false. Where both models fall short is their false positive churn predictions. Only half the predicted churn labels are true churns. This translates to both models often predicting false positives (FP).

Part 3: Permutation Importance

The last thing performed in “run_experiments.py” is calculating the permutation importance. It is a method that involves randomly distributing information in a feature. This allows us to calculate the performance of the model before and after the reshuffle, and gives a metric for how important the feature is for the model. If the performance does not decrease after shuffling a feature, the feature is not important for the model. Instead, if the performance degrades, the feature is important for the model.

To calculate permutation importance, the performance on the original data is first measured using accuracy as a metric. A random number generator specified by a seed was created, used to shuffle the features. This guarantees that differences in results are due to differences in the models themselves, rather than random variations in the shuffling process. The permutation was repeated 30 times on each feature, averaging the accuracy in permuted scores for each feature. The importance was determined by subtracting this average permuted score from the original score. The feature importances are visualized as bar plots in figure 4 and figure 5.

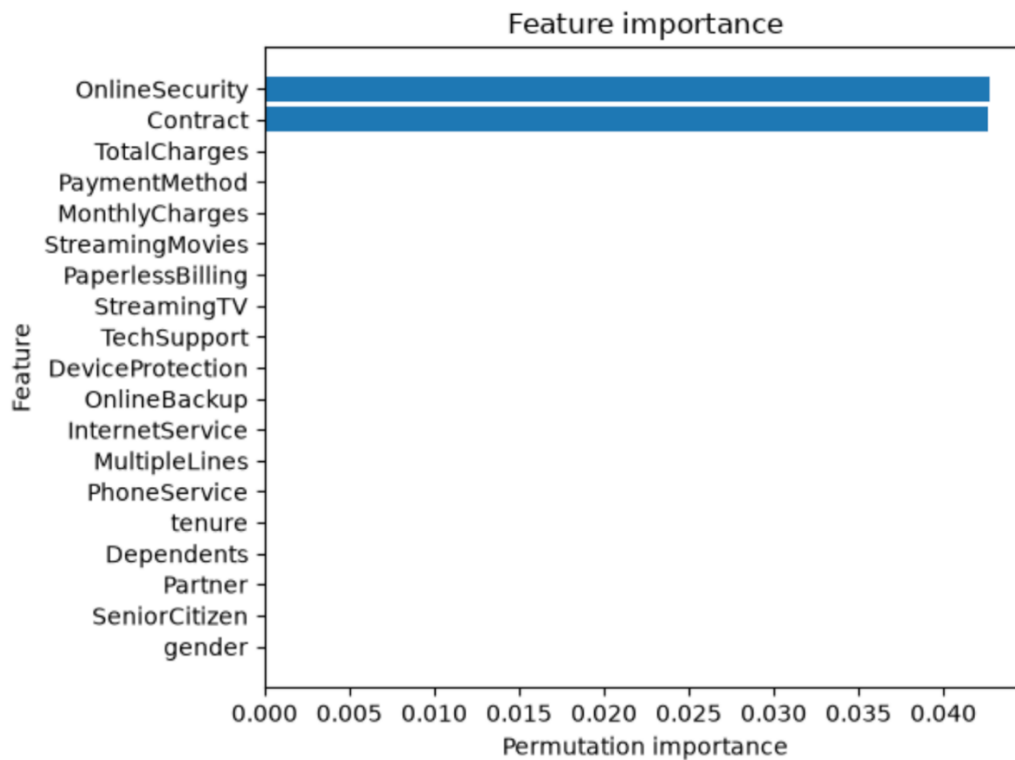


Figure 4: Feature importance in churn-data using permutation importance for the ID3-model.

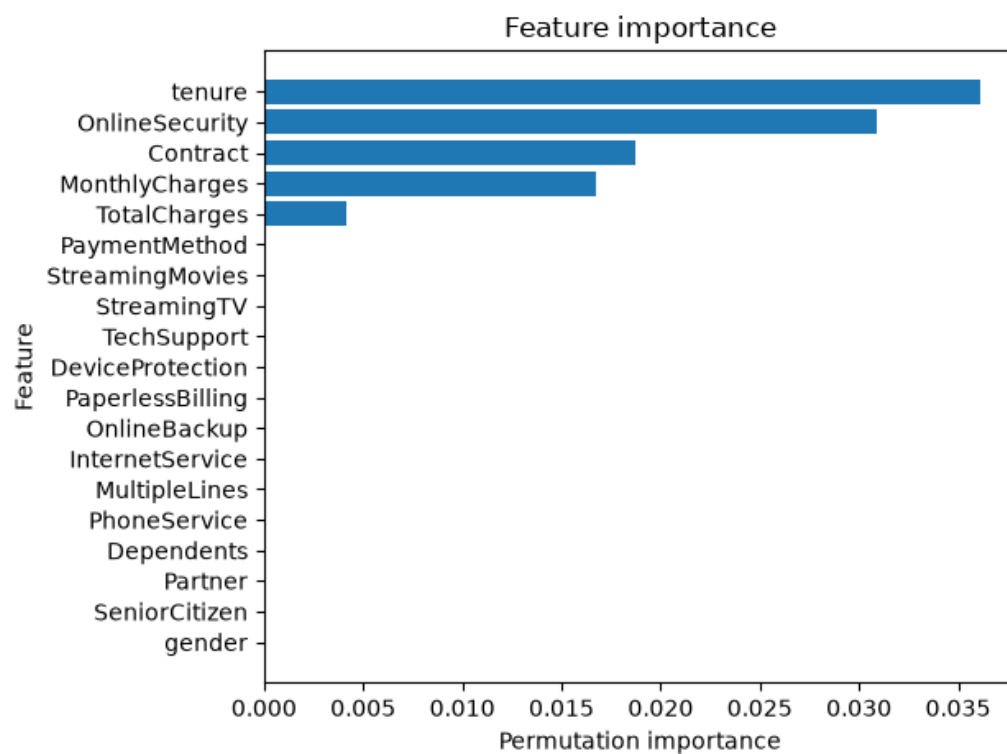


Figure 5: Feature importance in churn-data using permutation importance for the Scikit-learn model.

Figure 4 shows only two features being important for the model, *OnlineSecurity* and *Contract*. Both features drop the performance by about 0.043 points when shuffled. By the tree only having two layers, there are a maximum of tree features that can be split on. There only being two features in figure 4 indicate that one of the features was split on twice, for example in both the left and right node of the root node. The remaining features not having any impact on the performance, shows how the model does not consider these when classifying an instance.

In contrast, figure 5 shows five different features being important for the model, *tenure*, *OnlineSecurity*, *Contract*, *Monthly Charges* and *Total Charges*, with *tenure* being the most important. Scikit-learn built a deeper tree, with maximum depth of 4, allowing more features to be split on, resulting in more features being important to the model.

Compared to the ID3 model, *tenure* ranked as most important feature in the Scikit-learn model, followed by the key variables from the ID3 model: *OnlineSecurity* and *Contract*. This may be because continuous variables such as *tenure* require more precise splitting, which is more feasible at greater tree depths.

Figures 4 and 5 demonstrate that none of the models rely on demographic variables such as "gender" or "partner." This indicates that customer behavior and contract terms are more significant factors in predicting churn than the customer's personal characteristics.

Conclusion

Both the implemented ID3-model and the model of Scikit-learn yields relatively underwhelming results, with an F1-score at respectively $(0,59 \pm 0,02)$ and $(0,61 \pm 0,01)$. To improve predictions, a random forest model consisting of multiple trees could help detect complex patterns in the data and improve stability.

The Scikit-learn-model performs slightly better than the implemented model. This can be explained by the ID3-algorithm. The greedy algorithm will always favor the majority class, where as the algorithm of the Scikit-learn model can weigh different classes differently and be more optimal for the skewed churn-data dataset.

Furthermore, the overall low F1-scores can indicate that the data is missing crucial features to the prediction. There will almost certainly be features not collected that impact the prediction label. If the data available is missing some of these features, the data itself becomes a limiting factor to models performance, no matter how advanced it may be.

In conclusion, while the decision trees act as functioning classifiers, addressing data imbalances or building more complex models will be essential for achieving more reliable models.